Συστήματα Υπολογισμού Υψηλών Επιδόσεων
Project 2

Τσόγκας Παναγιώτης	**Νίκας Ιωάννης Ιάσων**	**Τσιαντός Δημήτριος**
ptsogkas@uth.gr	ioannikas@uth.gr	dtsiantos@uth.gr
3672	3771	3796

Νοέμβριος 2025

Περιεχόμενα

1 Εισαγωγή **3**
1.1 Ο αλγόριθμος . . . 3
1.2 Ψευδοκώδικας . . . 4
1.3 Το setup . . . 4

2 Παραλληλοποίηση **4**
2.1 Εξωτερικός βρόχος do-while . . . 4
2.2 Οι βασικοί βρόχοι . . . 4
2.3 Εύρεση απόστασης . . . 6
2.4 Αποτυχημένες Προσπάθειες . . . 6

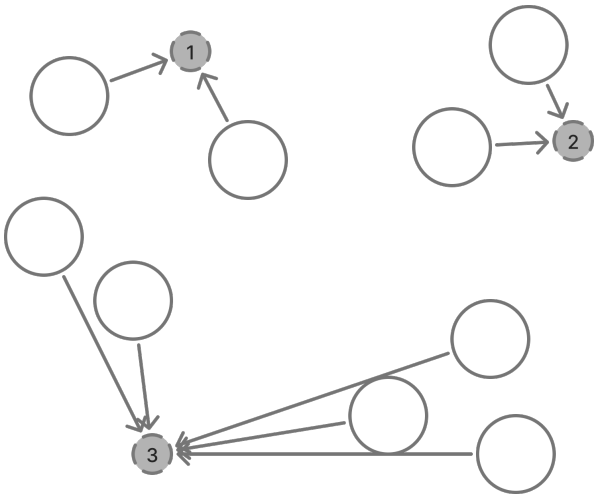
3 Χρόνοι εκτέλεσης **8**
3.1 Με parallel for και critical . . . 8
3.2 Με SIMD και critical . . . 9
3.3 Με SIMD και atomic . . . 10

1 Εισαγωγή

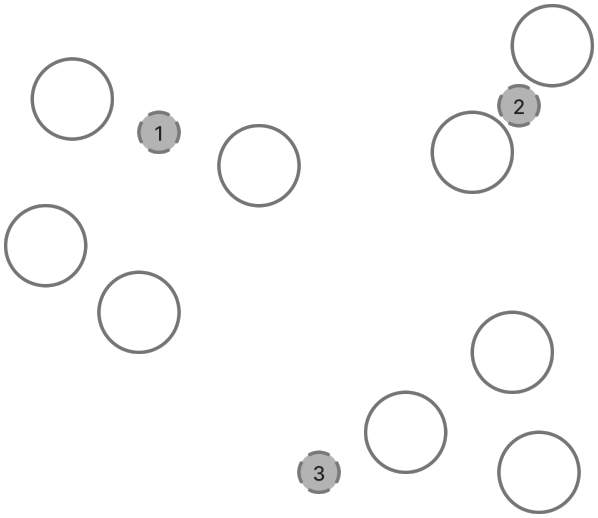
1.1 Ο αλγόριθμος

Ο k-means είναι ένας από τους πιο γνωστούς αλγόριθμους για ομαδοποίηση (clustering) δεδομένων. Στόχος του είναι να χωρίσει ένα σύνολο σημείων σε k ομάδες, έτσι ώστε τα σημεία κάθε ομάδας να είναι όσο το δυνατόν πιο κοντά στο κέντρο της (centroid) και όσο το δυνατόν πιο μακριά από τα κέντρα των άλλων ομάδων. Η διαδικασία επαναλαμβάνεται μέχρι να επιτευχθεί σύγκλιση, δηλαδή μέχρι τα κέντρα των ομάδων να σταθεροποιηθούν, ή να ξεπεραστεί το προκαθορισμένο όριο επαναλήψεων.

Στην υλοποίηση που μας ζητάται να παραλληλοποιήσουμε υπάρχουν δύο βασικοί βρόχοι. Στον πρώτο βρόχο υπολογίζεται για κάθε σημείο η απόστασή του από όλα τα κέντρα και το σημείο ανατίθεται στο κοντινότερο κέντρο. Στον δεύτερον βρόχο υπολογίζεται για κάθε ομάδα το νέο κέντρο ως ο μέσος όρος των σημείων της ομάδας.



Σχήμα 1: Παράδειγμα εκτέλεσης πρώτου βρόχου, αναθέτουμε το κάθε στοιχείο στο κοντινότερο κέντρο.



Σχήμα 2: Παράδειγμα εκτέλεσης δεύτερου βρόχου, επανυπολογίζουμε τα νέα κέντρα.

1.2 Ψευδοκώδικας

Algorithm 1 Αλγόριθμος k-means

Αρχικοποίηση: $membership[i] \leftarrow -1$ για κάθε σημείο i
repeat
  $\Delta \leftarrow 0$
  for κάθε αντικείμενο i **do** ▷ Πρώτος βρόχος
    $index \leftarrow$ find_nearest_cluster$(objects[i], clusters)$ ▷ Βρες το κοντινότερο κέντρο
    if $membership[i] \neq index$ **then**
      $\Delta \leftarrow \Delta + 1$ ▷ Ανανέωση Δ για έλεγχο σύγκλισης
    end if
    $membership[i] \leftarrow index$ ▷ Ανάθεση σε cluster
    $newClusterSize[index] \leftarrow newClusterSize[index] + 1$ ▷ Αύξηση πλήθους cluster
    for κάθε συντεταγμένη j **do**
      $newClusters[index][j] \leftarrow newClusters[index][j] + objects[i][j]$ ▷ Αύξηση αθροίσματος
    end for
  end for
  for κάθε κέντρο i **do** ▷ Δεύτερος βρόχος
    for κάθε συντεταγμένη j **do**
      if $newClusterSize[i] > 0$ **then**
        $clusters[i][j] \leftarrow \dfrac{newClusters[i][j]}{newClusterSize[i]}$ ▷ Εύρεση νέων κέντρων
      end if
      $newClusters[i][j] \leftarrow 0$
    end for
    $newClusterSize[i] \leftarrow 0$
  end for
  $\Delta \leftarrow \dfrac{\Delta}{numObjs}$
until $\Delta \leq threshold$ ή $loop \geq 500$ ▷ Έλεγχος σύγκλισης ή ανώτατου ορίου επαναλήψεων

1.3 Το setup

Για όλες τις δοκιμές και τα νούμερα χρησιμοποιήσαμε το CSL Venus με έκδοση icx Intel(R) oneAPI DPC++/C++ Compiler 2025.2.1 (2025.2.0.20250806) και τα ακόλουθα compiler flags:

```
-I.
-O3
-qopenmp
-DNDEBUG
-g
```

2 Παραλληλοποίηση

2.1 Εξωτερικός βρόχος do-while

Κοιτάζοντας την λογική του αλγορίθμου, καταλαβαίνουμε πως δεν μπορεί να παραλληλοποιηθεί το εξωτερικό do-while loop που εκτελεί τον kmeans πολλές φορές, καθώς υπάρχουν data dependencies από ένα iteration με το προηγούμενό του στις τιμές των centroids.

2.2 Οι βασικοί βρόχοι

Παρατηρούμε πως στον πρώτο βρόχο, στην εύρεση και ανάθεση των στοιχείων σε clusters, δεν υπάρχει κάποιο data dependency μεταξύ των επαναλήψεων. Για αυτό μπορούμε να χρησιμοποιήσουμε "`#pragma omp parallel for`".

```
#pragma omp parallel for private(j, index) reduction(+:delta)
for (i=0; i<numObjs; i++) {
    // ...

    /* update new cluster center : sum of objects located within */
    #pragma omp atomic
    newClusterSize[index]++;

    for (j=0; j<numCoords; j++) {
        #pragma omp atomic
        newClusters[index][j] += objects[i][j];
    }
}
```

Ύστερα από την ανάθεση του στοιχείου, ωστόσο, αυξάνεται το πλήθος των στοιχείων του cluster κατά ένα και αυξάνεται το συνολικό άθροισμα του cluster. Αυτές οι δύο εντολές χρειάζεται να γίνονται ατομικά από κάθε core, για αυτό και είναι απαραίτητη η χρήση ενός `#pragma omp atomic`. Δοκιμές έδειξαν πως στα 56 cores το `atomic` directive ήταν 27% γρηγορότερο σε σύγκριση με το `critical` directive. Αυτό συμβαίνει γιατί το `atomic` directive χρησιμοποιεί υποστήριξη από το υλικό, όπου υπάρχει, για να κάνει πράξεις ατομικά χωρίς να χρησιμοποιήσει συγχρονισμό. Στην περίπτωση μας θέλουμε να εκτελέσουμε ατομικά μια αύξηση κατά ένα και μια αύξηση κατά έναν αριθμό. Ειδικά για αυτές τις πράξεις είναι αρκετά σύνηθες να υπάρχει υποστήριξη από το υλικό για την ατομική τους εκτέλεση.

```
Before:

/* update new cluster center : sum of objects located within */
#pragma omp critical
{
    delta += local_delta;
    newClusterSize[index]++;
    for (j=0; j<numCoords; j++)
        newClusters[index][j] += objects[i][j];
}

After:

/* update new cluster center : sum of objects located within */
#pragma omp atomic
newClusterSize[index]++;
for (j=0; j<numCoords; j++) {
    #pragma omp atomic
    newClusters[index][j] += objects[i][j];
}
```

Πίνακας 1: Αποτελέσματα εκτέλεσης με critical

Threads	**Χρόνος (s)**	**Τυπική Απόκλιση**	**Speedup**
28	0.254	$\pm$ 0.023	14.23$\times$
56	0.263	$\pm$ 0.021	13.24$\times$

Αξίζει να σημειωθεί, επίσης, πως με την εντολή critical έχουμε χειρότερο χρόνο εκτέλεσης στα 56 threads σε σχέση με τα 28 threads. Αντίθετα, με την εντολή atomic τα 56 threads έχουν καλύτερο χρόνο από τα 28. Υποθέτουμε πως αυτή η επιβράδυνση στην εντολή critical οφείλεται στο γεγονός πως υπάρχει μεγαλύτερη πιθανότητα σε οποιαδήποτε χρονική στιγμή να βρίσκεται ένα thread στο κρίσιμο

Πίνακας 2: Αποτελέσματα εκτέλεσης με atomic

Threads	Χρόνος (s)	Τυπική Απόκλιση	Speedup
28	0.252	± 0.043	14.16×
56	0.206	± 0.009	17.31×

σημείο του κώδικα, οπότε αυξάνοντας τα threads αυξάνουμε τα νήματα που περιμένουν μέχρι να φύγει κάποιο άλλο νήμα από το κρίσιμο σημείο.

Όπως και στον πρώτο, στον δεύτερο βρόχο που υπολογίζονται τα νέα κέντρα, δεν υπάρχει κάποιο data dependency, άρα μπορεί να παραλληλοποιηθεί με το ίδιο directive.

```
/* average the sum and replace old cluster center with newClusters */
#pragma omp parallel for private(j)
for (i=0; i<numClusters; i++) {
    for (j=0; j<numCoords; j++) {
        if (newClusterSize[i] > 0)
            clusters[i][j] = newClusters[i][j] / newClusterSize[i];
        newClusters[i][j] = 0.0;   /* set back to 0 */
    }
    newClusterSize[i] = 0;   /* set back to 0 */
}
```

2.3 Εύρεση απόστασης

Μέσω profiling, χρησιμοποιώντας το Intel VTune Profiler, διαπιστώσαμε πως ένα από τα μεγαλύτερα hotspots ήταν στην συνάρτηση που υπολογίζει την ευκλείδια απόσταση μεταξύ δύο σημείων, καθώς καλείται πολλαπλές φορές κατά την διάρκεια μιας επανάληψης του πρώτου βρόχου. Κατά την προσπάθεια παραλληλοποίησης αυτής της συνάρτησης διαπιστώθηκε πως δεν προσδίδει επιτάχυνση αλλά αντιθέτως καταστρέφει εντελώς την απόδοση (βλέπε 2.4).

Για να πετύχουμε καλύτερη απόδοση στην συγκεκριμένη συνάρτηση χρησιμοποιήσαμε το directive "`#pragma omp simd`" το οποίο προσπαθεί να μετατρέψει τον βρόχο στον οποίο εφαρμόζεται σε βρόχο SIMD, δηλαδή πολλαπλές επαναλήψεις του βρόχου μπορούν να εκτελεστούν ταυτόχρονα με εντολές SIMD. Συγκεκριμένα ο Intel Xeon Processor E5-2695 v3 παρέχει AVX2 instruction set το οποίο χρησιμοποιεί 256-bit registers οπότε καταφέρνουμε να κάνουμε 8 float πράξεις ταυτόχρονα.

2.4 Αποτυχημένες Προσπάθειες

Κατά την ανάπτυξη του τελικού κώδικα, πραγματοποιήθηκαν αρκετές αποτυχημένες προσπάθειες παραλληλοποίησης. Σε μία απο αυτές, δοκιμάσαμε την παραλληλοποίηση της εσωτερικής συνάρτησης find_nearest_cluster με 56 threads, εφαρμόζοντας χειροκίνητα min reduction, καθώς οι παλαιότερες εκδόσεις της OpenMP δεν υποστηρίζουν reduction σε πίνακες. Ο κώδικας παρουσιάζεται εδώ:

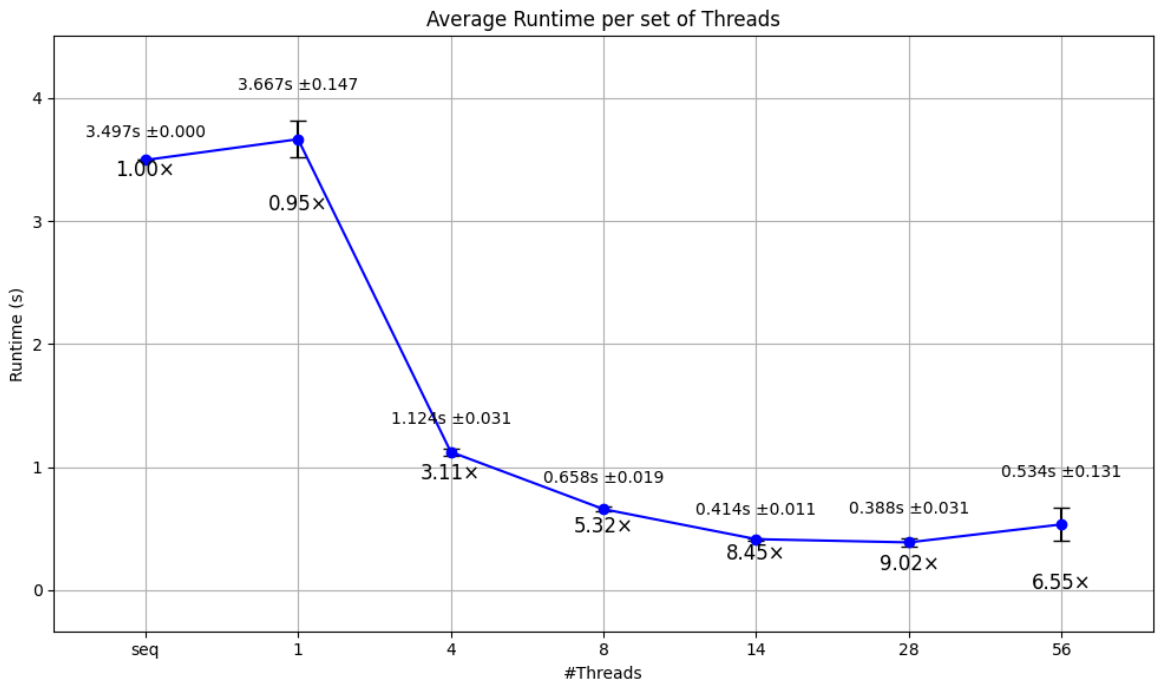
```
__inline static
int find_nearest_cluster2(int numClusters, int numCoords, float *object, float **clusters)
{
    float min_dist = FLT_MAX;
    int min_index = 0;

    #pragma omp parallel num_threads(56) // Set to 3 for nested approach
    {
        float local_min_dist = FLT_MAX;
        int local_index = 0;

        #pragma omp for nowait
        for (int i = 0; i < numClusters; i++) {
            float dist = euclid_dist_2(numCoords, object, clusters[i]);
            if (dist < local_min_dist) {
                local_min_dist = dist;
                local_index = i;
            }
        }

        #pragma omp critical
        {
            if (local_min_dist < min_dist) {
                min_dist = local_min_dist;
                min_index = local_index;
            }
        }
    }

    return min_index;
}
```

Ο χρόνος εκτέλεσης του προγράμματος ήταν αισθητά αυξημένος, με αυτή την μορφή παραλληλοποίησης, ειδικά σε σχέση με την σειριακή εκτέλεση. Παρά το γεγονός αυτο, δοκιμάσαμε μια προσέγγιση φωλιασμένης παραλληλοποίησης (nested parallelism), διαμοιράζοντας τα 56 threads του server σε διαφορετικά επίπεδα. Συγκεκριμένα, ο εξωτερικός (πρώτος) βρόχος παραλληλοποιήθηκε με 14 threads, ενώ στο εσωτερικό της συνάρτησης find_nearest_cluster χρησιμοποιήθηκαν 3 threads. Αφού έχουμε 3 thread για κάθε διεργασία, συνολικά έχουμε $14 + 14 \cdot 3 = 56$ threads. Για την ενεργοποίηση της φωλιασμένης παραλληλοποίησης χρησιμοποιήθηκε η συνάρτηση omp_set_max_active_levels(2).

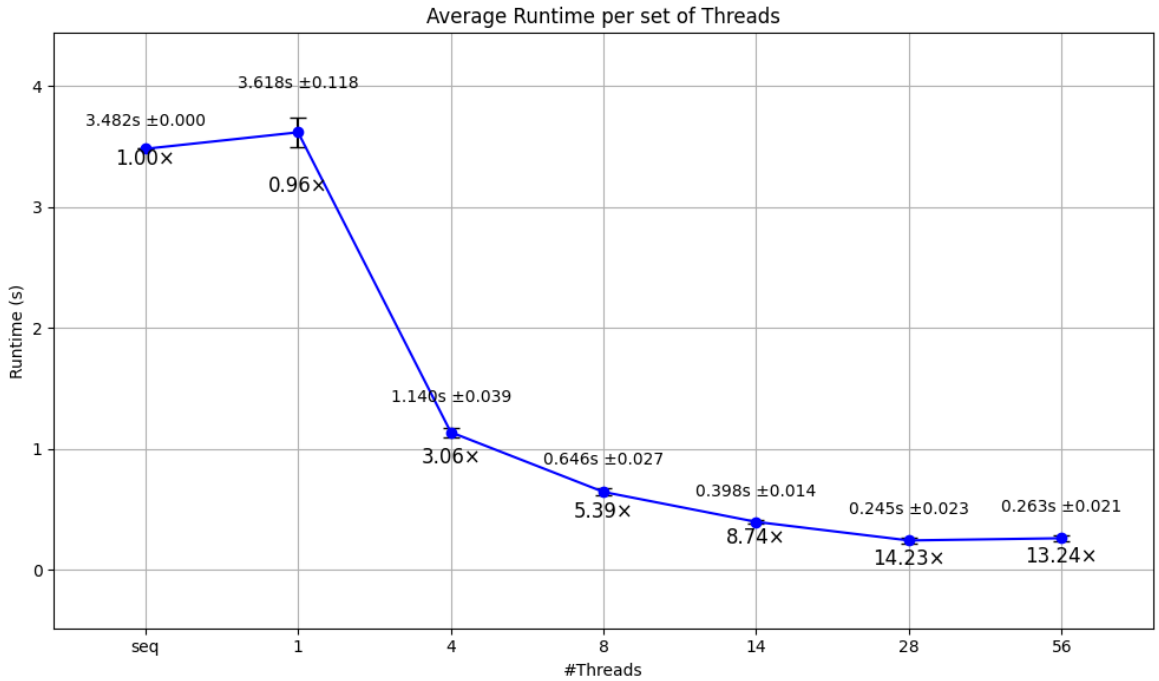
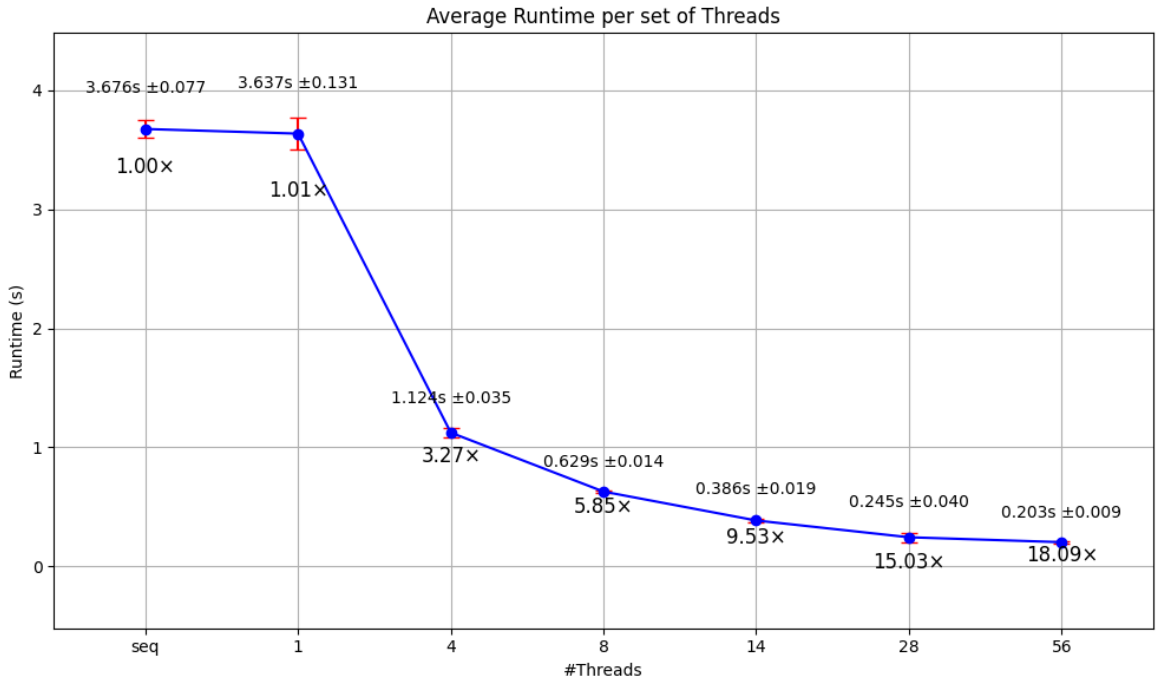
Παρ' όλα αυτά, τα αποτελέσματα ήταν παρόμοια, παρά την μείωση των thread απο 56 σε 3 στο εσωτερικό της συνάρτησης. Το αυξημένο overhead που προκύπτει από την δημιουργία και διαχείριση πολλαπλών επιπέδων νημάτων φαίνεται να υπερισχύει του οφέλους της φωλιασμένης παραλληλοποίησης. Για αυτόν τον λόγο, η ιδέα της φωλιασμένης παραλληλοποίησης απορρίφθηκε πλήρως και επιλέξαμε μια απλούστερη προσέγγιση.

3 Χρόνοι εκτέλεσης

3.1 Με parallel for και critical



3.2 Με SIMD και critical



3.3 Με SIMD και atomic